

SafeHer API

Test flow — 12 steps from registration to ride completion

Base URL: <http://localhost:5000/api/v1>

● Rider

● Driver

● Admin / DB

1

Register a rider

POST /auth/register

Endpoint

POST

/auth/register

Request body

```
{
  "name": "Test Rider",
  "email": "rider@test.com",
  "phone": "08012345678",
  "password": "password123",
  "role": "RIDER"
}
```

Expected response

```
{
  "success": true,
  "message": "Registration successful. Please verify your phone number.",
  "data": { "token": "eyJ...", "user": { "id": "uuid", "role": "RIDER" } }
}
```

Note: Save the token — use it for all rider requests.

2

Register a driver

POST /auth/register

Endpoint

POST

/auth/register

Request body

```
{
  "name": "Test Driver",
  "email": "driver@test.com",
  "phone": "08087654321",
  "password": "password123",
  "role": "DRIVER"
}
```

Expected response

```
{
  "success": true,
  "data": { "token": "eyJ...", "user": { "id": "uuid", "role": "DRIVER" } }
}
```

Note: Save the driver token separately.

3

Create driver profile

POST /drivers/register

Endpoint

POST /drivers/register

Headers

Authorization: Bearer <driver_token>

Request body

```
{
  "licenseNumber": "LAG12345",
  "licenseExpiry": "2027-01-01"
}
```

Expected response

```
{
  "success": true,
  "message": "Driver profile created",
  "data": { "id": "uuid", "verificationStatus": "PENDING" }
}
```

4

Register driver vehicle

POST /vehicles

Endpoint

POST /vehicles

Headers

Authorization: Bearer <driver_token>

Request body

```
{
  "plateNumber": "LAG-234-AB",
  "model": "Toyota Corolla",
  "color": "Silver",
  "year": 2020
}
```

Expected response

```
{
  "success": true,
  "data": { "id": "uuid", "plateNumber": "LAG-234-AB" }
}
```

5

Verify driver (DB)

Run SQL in Neon / Supabase

SQL

```
UPDATE "Driver"
SET "verificationStatus" = 'VERIFIED'
WHERE "userId" = (
  SELECT id FROM "User"
  WHERE email = 'driver@test.com'
);
```

Note: No API endpoint — run directly in your Neon/Supabase SQL editor.

6

Get fare estimate

GET /pricing

Endpoint

GET

/pricing?pickupLat=6.5244&pickupLng;=3.3792&destinationLat;=6.4550&destination
Lng;=3.3841

Headers

Authorization: Bearer <rider_token>

Expected response

```
{
  "success": true,
  "data": {
    "distanceKm": 7.6,
    "suggestedFare": 2950,
    "floorPrice": 2100,
    "message": "Minimum offer: NGN 2100. Suggested: NGN 2950"
  }
}
```

Note: Use suggestedFare and floorPrice to decide offeredFare in step 7.

7

Request a ride

POST /rides/request

Endpoint

POST

/rides/request

Headers

Authorization: Bearer <rider_token>

Request body

```
{
  "pickupAddress": "Victoria Island, Lagos",
  "pickupLat": 6.5244,
  "pickupLng": 3.3792,
  "destinationAddress": "Lekki Phase 1, Lagos",
  "destinationLat": 6.4550,
  "destinationLng": 3.3841,
  "offeredFare": 2500
}
```

Expected response

```
{
  "success": true,
  "data": {
    "ride": { "id": "<ride_id>", "status": "REQUESTED", "offeredFare": 2500 },
    "fareDetails": { "suggestedFare": 2950, "floorPrice": 2100 }
  }
}
```

Note: Save the ride id — needed for all remaining steps.

8

Test floor price validation (should fail)

POST /rides/request

Endpoint

POST /rides/request

Headers

Authorization: Bearer <rider_token>

Request body

```
{
  "pickupAddress": "Victoria Island, Lagos",
  "pickupLat": 6.5244,
  "pickupLng": 3.3792,
  "destinationAddress": "Lekki Phase 1, Lagos",
  "destinationLat": 6.4550,
  "destinationLng": 3.3841,
  "offeredFare": 500
}
```

Expected response

```
{
  "success": false,
  "message": "Your offer of NGN 500 is below the minimum allowed fare of NGN 2100 for this trip."
}
```

Note: Expected to fail — confirms server-side floor price enforcement.

9

Driver submits counter-offer

PATCH /rides/:id/counter

Endpoint

PATCH /rides//counter

Headers

Authorization: Bearer <driver_token>

Request body

```
{
  "counterFare": 3000
}
```

Expected response

```
{
  "success": true,
  "data": { "rideId": "<ride_id>", "offeredFare": 2500, "counterFare": 3000 }
}
```

10

Rider accepts counter-offer

PATCH /rides/:id/accept-offer

Endpoint

PATCH /rides//accept-offer

Headers

Authorization: Bearer <rider_token>

Expected response

```
{
  "success": true,
  "message": "Counter-offer accepted. Ride confirmed!",
  "data": { "agreedFare": 3000, "status": "ACCEPTED" }
}
```

11

Rider gets driver details

GET /rides/:id/driver-details

Endpoint

GET

/rides//driver-details

Headers

Authorization: Bearer <rider_token>

Expected response

```
{
  "success": true,
  "data": {
    "agreedFare": 3000,
    "driver": {
      "name": "Test Driver",
      "phone": "08087654321",
      "averageRating": 0,
      "vehicle": { "model": "Toyota Corolla", "color": "Silver", "plateNumber": "LAG-234-AB" }
    }
  }
}
```

12

Complete the ride

PATCH /rides/:id/complete

Endpoint

PATCH

/rides//complete

Headers

Authorization: Bearer <driver_token>

Request body

```
{
  "fare": 3000,
  "distanceKm": 7.6,
  "durationMinutes": 25
}
```

Expected response

```
{
  "success": true,
  "data": { "status": "COMPLETED", "fare": 3000, "agreedFare": 3000 }
}
```

Note: Ride must be in TRIP_STARTED status. Walk through /arrived and /start first if needed.